

Programação Orientada a Objetos

Questões para a prova

FURB · 2026/02 · Professor André Felipe Bürger

88 questões respondidas, em duas partes.

Parte I — Lista de revisão do professor (L07). As 30 questões da lista, na íntegra, com a alternativa correta marcada e a explicação.

Parte II — Questões que podem cair. 58 questões extras escritas a partir do material da disciplina, nos mesmos três formatos: assinalar, somatória e relacionar colunas. Cobrem os mesmos assuntos da revisão e mais associações, multiplicidade e ArrayList, que a lista não cobre.

Ao fim de cada parte há o gabarito em tabela, para conferência rápida.

Índice

Parte I — Lista de revisão (L07)

Paradigmas e conceitos básicos de POO	Q1 – Q5
UML e diagrama de classes	Q6 – Q9
Implementação, memória e diagrama de objetos	Q10 – Q13
Escopo de variáveis	Q14 – Q16
Encapsulamento	Q17 – Q20
Membros de classe, sobrecarga e construtores	Q21 – Q24
Lançamento e tratamento de exceções	Q25 – Q29
Qualidade de software e testes de unidade	Q30 – Q30

Parte II — Questões que podem cair

Paradigmas e conceitos básicos de POO	B01 – B06
UML: diagrama de classes e diagrama de objetos	B07 – B11
Implementação, memória e o operador new	B12 – B15
Escopo de variáveis e inicialização	B16 – B20
Encapsulamento	B21 – B25
Membros de classe, sobrecarga e construtores	B26 – B30
Lançamento de exceções	B31 – B34
Tratamento de exceções	B35 – B41
Qualidade de software e testes de unidade	B42 – B48
Associações, multiplicidade e ArrayList	B49 – B58

Parte I — Lista de revisão do professor (L07)

Paradigmas e conceitos básicos de POO

Q1 assinalar

Um paradigma de programação determina a visão que o programador tem sobre a estruturação e a execução do programa. Sobre o paradigma orientado a objetos, assinale a alternativa correta:

- a) Trata a computação como avaliação de funções matemáticas, evitando estados e dados mutáveis.
- b) Baseia-se exclusivamente em fatos e regras que descrevem relações entre dados.
- ✓ c) **Organiza o código em objetos, que são instâncias de classes, combinando dados e comportamentos.**
- d) Baseia-se apenas em comandos sequenciais que modificam o estado do programa, sem noção de objeto.
- e) É um paradigma que não pode ser aplicado à linguagem Java.

RESPOSTA c

POO organiza o programa em objetos, que são instâncias de classes e juntam dados (atributos) com comportamento (métodos). As outras alternativas descrevem funcional (a), lógico (b) e imperativo (d).

Q2 relacionar colunas

Relacione cada paradigma de programação (coluna da esquerda) à sua característica (coluna da direita):

Coluna da esquerda: A. Imperativo · B. Funcional · C. Lógico · D. Orientado a objetos

- (C) Programas consistem em fatos e regras (ex.: Prolog).
- (D) Empacota dados e comportamento em objetos (ex.: Java).
- (A) Comandos sequenciais que alteram o estado (ex.: C, Pascal).
- (B) Avaliação de funções matemáticas, evitando estado mutável (ex.: Haskell).

RESPOSTA C - D - A - B

Prolog é lógico, Java é orientado a objetos, C e Pascal são imperativos, Haskell é funcional.

Q3 somatória

Sobre os objetivos e as contrapartidas do paradigma orientado a objetos, some os números das afirmativas CORRETAS:

- ✓ 01. **Um dos objetivos é melhorar a compreensão do sistema e facilitar sua manutenção e evolução.**
- ✓ 02. **Busca alto grau de reutilização por meio de componentes acopláveis.**
- 04. Garante que qualquer programa OO será sempre menor que o equivalente imperativo.
- ✓ 08. **Tem como contrapartida uma maior curva de aprendizagem.**
- 16. É sempre a melhor escolha para qualquer tipo de problema, sem exceção.

RESPOSTA 11 (01 + 02 + 08)

A 04 e a 16 são exageros: POO não garante código menor e não é a melhor escolha para todo problema. Reutilização e curva de aprendizagem são, respectivamente, objetivo e contrapartida.

Q4 somatória

Considerando os conceitos de classe, objeto, atributo e estado, some os números das afirmativas CORRETAS:

- ✓ 01. **Um objeto normalmente representa um elemento do mundo real e pertence a uma classe.**
- ✓ 02. **Uma classe descreve as características comuns de um grupo de objetos e funciona como um modelo para criá-los.**
- ✓ 04. **O estado de um objeto corresponde ao valor de seus atributos, e esse estado pode mudar ao longo do tempo.**
- ✓ 08. **O valor de um atributo é um dado que pertence apenas àquele objeto.**
- 16. Objetos possuem apenas atributos; não são capazes de executar operações.

RESPOSTA 15 (01 + 02 + 04 + 08)

A 16 é a falsa: objeto tem atributos E operações. O resto é a definição padrão de classe, objeto, atributo e estado.

Q5 assinalar

Assinale a alternativa que descreve corretamente o papel de uma classe em POO:

- a) A classe armazena os valores concretos de um único objeto em tempo de execução.
- ✓ b) **A classe define quais atributos os objetos podem possuir e quais operações podem realizar.**
- c) A classe é uma fotografia do estado do sistema em um instante específico.
- d) A classe só existe depois que o operador new é chamado.
- e) A classe é sinônimo de variável de referência.

RESPOSTA b

A classe é o molde: define os atributos e as operações. Quem guarda valores concretos é o objeto (a), e a fotografia do sistema é o diagrama de objetos (c).

UML e diagrama de classes

Q6 somatória

Sobre a UML e o diagrama de classes, some os números das afirmativas CORRETAS:

- ✓ 01. **A UML é uma linguagem visual para modelar sistemas sob o paradigma de orientação a objetos.**
- ✓ 02. **O diagrama de classes modela uma visão estática do software, mostrando classes e seus relacionamentos.**
- 04. É obrigatório modelar todo sistema utilizando os 14 diagramas da UML.
- ✓ 08. **Nos nomes de classes, atributos e operações não se utilizam espaços.**
- ✓ 16. **A UML é a linguagem padrão de modelagem de software adotada internacionalmente pela indústria.**

RESPOSTA 27 (01 + 02 + 08 + 16)

A 04 é a falsa: usam-se os diagramas necessários, não os 14. As demais são a definição de UML, do diagrama de classes e da convenção de nomes sem espaço.

Q7 relacionar colunas

Relacione o elemento ao estilo/convenção de nomenclatura correto:

Coluna da esquerda: A. Nome de classe · B. Nome de atributo · C. Nome de operação · D. Tipo de dado

- (C) lowerCamelCase; é um verbo ou locução verbal, com parênteses após o nome.
- (D) delimita os valores possíveis (ex.: int, double, boolean, String).
- (A) UpperCamelCase; substantivo do vocabulário do sistema (ex.: ContaBancaria).
- (B) lowerCamelCase; substantivo que representa uma propriedade da classe.

RESPOSTA C - D - A - B

Classe é UpperCamelCase e substantivo; atributo é lowerCamelCase e substantivo; operação é lowerCamelCase, verbo e com parênteses; tipo de dado delimita os valores possíveis.

Q8 relacionar colunas

Relacione cada tipo de dado Java à sua descrição:

Coluna da esquerda: A. boolean · B. int · C. double · D. String

- (C) Representa números decimais.
- (D) Representa texto.
- (A) Tipo lógico, com valores verdadeiro ou falso.
- (B) Representa números inteiros.

RESPOSTA C - D - A - B

double é decimal, String é texto, boolean é lógico, int é inteiro.

Q9 assinalar

Sobre as operações de uma classe, assinale a alternativa correta:

- a) Toda operação obrigatoriamente altera o estado do objeto.
- ✓ **b) Uma operação pode alterar o estado do objeto ou, a partir do estado, produzir um valor de retorno.**
- c) O nome de uma operação deve ser um substantivo em UpperCamelCase.
- d) Operações não podem receber parâmetros.
- e) Operações e atributos são a mesma coisa em UML.

RESPOSTA b

Operação pode ser procedimento (altera o estado, retorna void) ou função (calcula a partir do estado e devolve valor). Nome de operação é verbo em lowerCamelCase, não substantivo.

Implementação, memória e diagrama de objetos

Q10 somatória

Sobre a tradução de um diagrama de classes para código Java, some os números das afirmativas CORRETAS:

- ✓ **01. Os atributos da classe geralmente são traduzidos em variáveis de instância.**
- ✓ **02. As operações da classe são traduzidas em métodos.**
- ✓ **04. Um método que apenas altera o estado do objeto e não devolve valor deve declarar o tipo de retorno void.**
- ✓ **08. Um método que devolve um valor deve declarar o tipo do dado devolvido.**

16. Métodos nunca podem usar o valor dos atributos do próprio objeto para realizar cálculos.

RESPOSTA 15 (01 + 02 + 04 + 08)

A 16 é a falsa: usar os próprios atributos em cálculos é exatamente o que os métodos fazem. O resto é a tradução direta diagrama → Java.

Q11 assinalar — leitura de código

Considere o trecho abaixo, em que a classe Pessoa possui os atributos públicos altura e peso e o método calcularImc():

```
Pessoa marta = new Pessoa();  
marta.altura = 1.70;  
marta.peso = 79;  
double imc = marta.calcularImc();
```

Assinale a alternativa correta sobre esse código:

- ✓ **a) O operador de membro (ponto) é usado para acessar atributos e chamar métodos do objeto.**
- b) A palavra new não é necessária para criar o objeto marta.
- c) A variável marta armazena diretamente os valores dos atributos, não um endereço.
- d) O método calcularImc() precisa receber altura e peso como parâmetros para funcionar.
- e) A linha "double imc = marta.calcularImc();" provoca erro de compilação.

RESPOSTA a

O ponto é o operador de acesso a membro. new é obrigatório (b), a variável guarda o endereço e não os valores (c), e o método usa os atributos do próprio objeto, sem precisar de parâmetros (d).

Q12 somatória

Sobre variáveis de referência, o operador new e as áreas de memória do Java, some os números das afirmativas CORRETAS:

- ✓ 01. Uma variável de referência armazena o endereço de um objeto, e não o objeto em si.
- ✓ 02. A área Heap armazena os objetos.
- ✓ 04. A área Stack armazena variáveis locais, parâmetros e chamadas de métodos.
- ✓ 08. O operador new cria o objeto na memória, inicializa as variáveis de instância e retorna o endereço criado.

16. O operador new é usado apenas para tipos primitivos como int e double.

RESPOSTA 15 (01 + 02 + 04 + 08)

A 16 é a falsa: new é para objetos; int e double são primitivos e não usam new. Heap guarda objetos, Stack guarda variáveis locais, parâmetros e chamadas.

Q13 assinalar

Sobre o diagrama de objetos, assinale a alternativa correta:

- ✓ a) **Mostra uma fotografia do estado do sistema em um determinado instante e contém somente objetos.**
- b) Substitui completamente o diagrama de classes, pois contém as mesmas informações.
- c) Contém apenas classes e seus relacionamentos, sem objetos.
- d) É usado para modelar o comportamento dinâmico de métodos ao longo do tempo.
- e) Não pode representar os valores dos atributos de um objeto.

RESPOSTA a

Diagrama de objetos é a fotografia do sistema num instante, com objetos e os valores dos atributos. Ele complementa o diagrama de classes, não substitui.

Escopo de variáveis

Q14 relacionar colunas

Relacione cada tipo de variável ao seu escopo:

Coluna da esquerda: A. Variável de instância · B. Variável local · C. Variável de bloco

- (C) Seu escopo é o bloco (ex.: dentro de um do/while) em que foi declarada.
- (A) Seu escopo são todos os métodos da classe.
- (B) Seu escopo é o próprio método em que foi declarada.

RESPOSTA C - A - B

Bloco vale só dentro do bloco; instância vale em todos os métodos da classe; local vale só no método onde foi declarada.

Q15 somatória

Sobre inicialização e valores padrão de variáveis em Java, some os números das afirmativas CORRETAS:

- ✓ 01. **Variáveis locais não possuem valor inicial; só podem ser lidas após atribuição explícita.**
- ✓ 02. **Variáveis de instância numéricas são inicializadas automaticamente com 0.**
- ✓ 04. **Variáveis de instância booleanas são inicializadas automaticamente com false.**
- ✓ 08. **Variáveis de instância de referência são inicializadas automaticamente com null.**

16. Variáveis locais recebem os mesmos valores padrão das variáveis de instância.

RESPOSTA 15 (01 + 02 + 04 + 08)

A 16 é a falsa: variável local NÃO recebe valor padrão. Só as de instância recebem (0, false, null).

Considere a classe abaixo:

```
public class Classe1 {  
    int var1 = 20;  
    void exibir() {  
        int var1;  
        this.var1 = 10;  
        System.out.println(var1);  
    }  
}
```

Ao chamar o método exibir(), o que será impresso e por quê? Assinale a alternativa correta:

- a) Imprime 10, porque this.var1 e var1 local são a mesma variável.
- b) Imprime 20, porque this.var1 sobrescreve a variável local.
- ✓ c) **A saída é indefinida (lixo), pois a variável local var1 não foi inicializada antes do println.**
- d) Erro de compilação, pois não é permitido repetir o nome var1.
- e) Imprime 0, pois toda variável local começa em zero.

RESPOSTA c

A local var1 foi declarada e nunca inicializada, e é ela que o println lê — this.var1 = 10 mexe na variável de instância, que é outra. **ATENÇÃO:** o compilador Java recusa esse código com "variable var1 might not have been initialized"; veja a observação no fim deste arquivo.

Encapsulamento

Uma classe ContaBancaria possui o atributo saldo com acesso público. Em outra classe, é possível escrever: conta.saldo = 10000; sem realizar um depósito. Assinale a alternativa que explica corretamente o problema e a solução:

- a) Não há problema, pois atributos públicos garantem a integridade dos dados.
- ✓ b) **O problema é o acesso direto ao atributo; a solução é encapsular (tornar saldo privado) e controlar o acesso por métodos.**
- c) A solução é declarar o atributo saldo como static.
- d) O problema só ocorre se a classe não tiver construtor.
- e) A solução é remover o atributo saldo da classe.

RESPOSTA b

Atributo público deixa qualquer classe alterar o saldo sem passar pelas regras. A solução é encapsular: private + métodos que validam (depositar/sacar).

Relacione o símbolo de visibilidade da UML à palavra reservada / significado em Java:

Coluna da esquerda: A. - (menos) · B. + (mais) · C. # (sustenido)

- (B) public — visível para qualquer classe.
- (C) protected — protegido.
- (A) private — somente visível pela própria classe.

RESPOSTA B - C - A

Menos é private, mais é public, sustenido é protected.

Q19 somatória

Sobre métodos de acesso (getters e setters), some os números das afirmativas CORRETAS:

- ✓ 01. Um getter recupera o valor de um atributo; nunca tem parâmetro e sempre retorna um valor.
- ✓ 02. Um setter atribui valor a um atributo; sempre tem um parâmetro e normalmente é do tipo void (procedimento).
- ✓ 04. Para um atributo booleano, o getter pode usar o prefixo "is" em vez de "get".
- ✓ 08. O tipo de retorno de um getter deve ser igual ao tipo do atributo correspondente.

16. Getters e setters devem ser sempre privados para funcionar.

RESPOSTA 15 (01 + 02 + 04 + 08)

A 16 é a falsa: getter e setter existem justamente para serem públicos. O resto é a regra dos métodos de acesso, incluindo o prefixo is para boolean.

Q20 assinalar — leitura de código

Considere o setter abaixo:

```
private String titular;  
public void setTitular(String titular) {  
    this.titular = titular;  
}
```

Qual o papel da palavra this nesse código? Assinale a alternativa correta:

- a) this é opcional e não faz diferença alguma nesse caso.
- ✓ b) this.titular refere-se à variável de instância, distinguindo-a do parâmetro de mesmo nome.
- c) this.titular refere-se ao parâmetro recebido pelo método.
- d) this transforma o atributo titular em uma variável de classe (static).
- e) this provoca erro de compilação quando o parâmetro tem o mesmo nome do atributo.

RESPOSTA b

Quando o parâmetro tem o mesmo nome do atributo, ele sombreia o atributo dentro do método. this.titular desfaz a ambiguidade e aponta para a variável de instância.

Membros de classe, sobrecarga e construtores

Q21 somatória

Sobre membros de classe (estáticos), some os números das afirmativas CORRETAS:

- ✓ 01. Uma variável de classe (static) é comum e compartilhada entre todas as instâncias.
- ✓ 02. Membros de classe podem ser usados sem que haja uma instância da classe.
- ✓ 04. Um método de classe (static) não pode utilizar a palavra this.
- ✓ 08. No diagrama de classes, os membros de classe são sublinhados.

16. Um método static pode manipular variáveis de instância livremente, sem uma instância explícita.

RESPOSTA 15 (01 + 02 + 04 + 08)

A 16 é a falsa: método static não tem objeto, então não acessa variável de instância sem uma instância explícita — e por isso também não pode usar this.

Q22 assinalar

Assinale a alternativa que contém apenas exemplos de membros estáticos (de classe), acessados pela sintaxe `classe.identificador`:

- ✓ a) **`Math.sqrt()`, `Math.abs()`, `Integer.MAX_VALUE`**
- b) `new Pessoa()`, `conta.sacar()`, `marta.altura`
- c) `this.titular`, `this.saldo`, `this.nome`
- d) `getTitular()`, `setTitular()`, `depositar()`
- e) `try`, `catch`, `finally`

RESPOSTA a

`Math.sqrt()`, `Math.abs()` e `Integer.MAX_VALUE` são acessados pelo nome da classe, sem instância. As outras alternativas usam objetos (b, c), métodos de instância (d) ou palavras reservadas (e).

Q23 somatória

Sobre a sobrecarga de métodos em Java, some os números das afirmativas CORRETAS:

- ✓ 01. **Sobrecarga é a implementação de vários métodos com o mesmo nome na mesma classe.**
- ✓ 02. **Métodos sobrecarregados devem ter assinaturas diferentes (lista de parâmetros diferente).**
- 04. O compilador diferencia métodos sobrecarregados apenas pelo tipo de retorno.
- ✓ 08. **Dois métodos com a mesma assinatura mas retornos diferentes não podem coexistir na mesma classe.**
- ✓ 16. **A sobrecarga deve ser usada com moderação, pois o excesso pode reduzir a legibilidade.**

RESPOSTA 27 (01 + 02 + 08 + 16)

A 04 é a falsa: o tipo de retorno NÃO diferencia sobrecarga. O que diferencia é a assinatura, isto é, a lista de parâmetros.

Q24 somatória

Sobre construtores, some os números das afirmativas CORRETAS:

- ✓ 01. **Um construtor é invocado exclusivamente durante a criação de objetos e serve para inicializá-los.**
- ✓ 02. **O construtor não possui tipo de dado de retorno e tem o mesmo identificador da classe.**
- ✓ 04. **Se nenhum construtor for implementado, o compilador fornece um construtor padrão (sem argumentos).**
- ✓ 08. **O operador `new`, ao criar o objeto, executa o construtor utilizado.**
- 16. Todo construtor obrigatoriamente deve declarar `void` como tipo de retorno.

RESPOSTA 15 (01 + 02 + 04 + 08)

A 16 é a falsa: construtor não declara tipo de retorno nenhum, nem `void`. Se declarar, vira um método comum com o nome da classe.

Lançamento e tratamento de exceções

Q25 assinalar

Assinale a alternativa que define corretamente o que é uma exceção:

- ✓ a) **Um evento que ocorre durante a execução do programa e interrompe o fluxo normal de execução.**
- b) Um erro de digitação detectado apenas pelo editor de texto.
- c) Um tipo primitivo do Java usado para representar números muito grandes.
- d) Um comentário especial que documenta o código.
- e) Uma variável de classe compartilhada entre instâncias.

RESPOSTA a

Exceção é um evento em tempo de execução que interrompe o fluxo normal do programa.

Q26 assinalar — leitura de código

Considere o método abaixo:

```
public void setSalario(double novoSalario) {  
    if (novoSalario < 0) {  
        throw new IllegalArgumentException("Salário incorreto");  
    }  
    salario = novoSalario;  
}
```

Assinale a alternativa correta sobre esse código:

- ✓ a) **Se novoSalario for negativo, uma exceção é lançada e a atribuição a salario não ocorre.**
- b) A palavra throw serve para capturar (tratar) a exceção.
- c) O método sempre lança a exceção, independentemente do valor de novoSalario.
- d) A mensagem "Salário incorreto" não pode ser recuperada no tratamento da exceção.
- e) O código não compila, pois throw não existe em Java.

RESPOSTA a

O throw interrompe o método na hora, então a linha de atribuição não chega a executar. throw lança; quem captura é o catch.

Q27 somatória

Sobre o comando try.catch e o tratamento de exceções, some os números das afirmativas CORRETAS:

- ✓ 01. **Qualquer erro em um comando do bloco try desvia o fluxo para o bloco catch.**
- ✓ 02. **Se nenhum erro ocorrer no try, os comandos do catch não são executados.**
- ✓ 04. **O comando catch exige um parâmetro que pode ser qualquer classe que estenda de Throwable.**
- ✓ 08. **Em múltiplos catch, as cláusulas são verificadas na ordem em que foram escritas.**

16. A cláusula finally só é executada quando ocorre um erro no bloco try.

RESPOSTA 15 (01 + 02 + 04 + 08)

A 16 é a falsa: finally executa sempre, com ou sem erro. Veja a ressalva sobre a afirmativa 01 no fim deste arquivo.

Q28 relacionar colunas

Relacione cada conceito de tratamento de exceções à sua descrição:

Coluna da esquerda: A. throws · B. finally · C. Exceção verificada · D. Classe Error

- (B) Bloco cujos comandos sempre executam, com ou sem erro (ex.: liberar recursos).
- (D) Indica erros graves, que não se espera que sejam tratados pelo programa.
- (A) Cláusula que delega ao método chamador o tratamento de uma exceção.
- (C) Deve ser tratada ou postergada; não é subclasse de RuntimeException nem de Error.

RESPOSTA B - D - A - C

finally sempre executa; Error é erro grave que não se espera tratar; throws delega ao chamador; exceção verificada é a que o compilador obriga a tratar ou declarar.

Q29 somatória

Sobre a hierarquia de erros, propagação e cláusulas catch, some os números das afirmativas CORRETAS:

- ✓ 01. A classe **Exception** e suas subclasses indicam erros que poderiam ser tratados pelos programas.
- ✓ 02. Se um método não trata o erro, ele é propagado ao método chamador, até chegar em **main()**.
- ✓ 04. Uma exceção pode ser capturada por um catch cujo parâmetro seja da mesma classe ou de uma superclasse do erro (polimorfismo).
- ✓ 08. A partir do Java 7, um único catch pode tratar várias classes de erro usando o operador **|** (ex.: **IOException | SQLException**).

16. É válido declarar um catch de uma subclasse depois de um catch de sua superclasse (ex.: **Exception** antes de **NumberFormatException**).

RESPOSTA 15 (01 + 02 + 04 + 08)

A 16 é a falsa: catch de subclasse depois da superclasse é código inalcançável e não compila. A ordem certa é da mais específica para a mais genérica.

Qualidade de software e testes de unidade

Q30 somatória

Sobre qualidade de software, testes de unidade e JUnit, some os números das afirmativas CORRETAS:

- ✓ 01. Quanto mais cedo os erros são detectados no ciclo de desenvolvimento, menor o custo de correção.
- ✓ 02. Em POO, a "unidade" testada em um teste de unidade é o método.
- ✓ 04. No JUnit, um método de teste recebe a anotação **@Test**, é público, não tem parâmetros e não retorna valor (**void**).
- ✓ 08. **assertEquals** verifica se o valor esperado é igual ao valor obtido; **assertThrows** verifica se uma exceção esperada é lançada.

16. Um teste de unidade avalia o sistema inteiro sob o ponto de vista do usuário final, em ambiente de produção.

RESPOSTA 15 (01 + 02 + 04 + 08)

A 16 é a falsa: avaliar o sistema inteiro pelo olhar do usuário final é teste de sistema/aceitação, não de unidade.

Gabarito — Parte I

Q	Resposta	Q	Resposta	Q	Resposta	Q	Resposta
Q1	c	Q9	b	Q17	b	Q25	a
Q2	C - D - A - B	Q10	15 (01 + 02 + 04 + 08)	Q18	B - C - A	Q26	a
Q3	11 (01 + 02 + 08)	Q11	a	Q19	15 (01 + 02 + 04 + 08)	Q27	15 (01 + 02 + 04 + 08)
Q4	15 (01 + 02 + 04 + 08)	Q12	15 (01 + 02 + 04 + 08)	Q20	b	Q28	B - D - A - C
Q5	b	Q13	a	Q21	15 (01 + 02 + 04 + 08)	Q29	15 (01 + 02 + 04 + 08)
Q6	27 (01 + 02 + 08 + 16)	Q14	C - A - B	Q22	a	Q30	15 (01 + 02 + 04 + 08)
Q7	C - D - A - B	Q15	15 (01 + 02 + 04 + 08)	Q23	27 (01 + 02 + 08 + 16)		
Q8	C - D - A - B	Q16	c	Q24	15 (01 + 02 + 04 + 08)		

Duas ressalvas importantes

Questão 16 — o compilador não deixa esse código rodar

A alternativa (c) é a que aponta a causa certa: a variável local `var1` nunca recebeu valor, e é ela que o `println` lê. Mas em Java de verdade o código nem compila:

```
error: variable var1 might not have been initialized
    System.out.println(var1);
```

Ou seja, a conclusão da alternativa (d) — erro de compilação — está certa, só que pelo motivo errado: repetir o nome é permitido, isso se chama **sombreamento**. Se a prova perguntar o que o compilador faz, a resposta é erro de compilação por variável local não inicializada. Se perguntar qual variável o `println` lê, é a local.

Questão 27, afirmativa 01 — "qualquer erro desvia para o catch"

Vale como correta na leitura da aula, mas o desvio só acontece se existir um `catch` cujo parâmetro seja da mesma classe do erro ou de uma superclasse dele. Sem `catch` compatível, a exceção é **propagada ao método chamador** — que é exatamente o que a afirmativa 02 da questão 29 diz.

Padrão das somatórias

Em todas as somatórias desta lista a afirmativa **16 é a incorreta** — ela sempre exagera ("garante", "sempre", "nunca", "livremente", "apenas"). Nas questões 3, 6 e 23 há uma segunda incorreta no meio da lista (a 04, nas três). Isso dá o padrão: **15** quando só a 16 é falsa, **11** ou **27** quando a 04 também é.

Parte II — Questões que podem cair

Paradigmas e conceitos básicos de POO

B01 assinalar

O mesmo cálculo de Fibonacci foi escrito em Pascal, Haskell, Prolog e Java no material da disciplina. Sobre o que essa comparação demonstra, assinale a alternativa correta:

- a) Que apenas o paradigma orientado a objetos é capaz de resolver o problema.
- ✓ **b) Que o paradigma determina a visão que o programador tem sobre a estruturação e a execução do programa, e um mesmo problema pode ser resolvido em vários paradigmas.**
- c) Que linguagens multiparadigma não existem na prática.
- d) Que o paradigma lógico é sempre mais eficiente que o imperativo.
- e) Que o paradigma escolhido não influencia a forma como o código é escrito.

RESPOSTA b

Um paradigma fornece e determina a visão sobre a estruturação e a execução do programa. Algumas linguagens são multiparadigma, como Python, que aparece citada tanto no imperativo quanto no orientado a objetos.

B02 somatória

Sobre as contrapartidas do paradigma orientado a objetos apontadas no material, some os números das afirmativas CORRETAS:

- ✓ **01. Maior curva de aprendizagem.**
- ✓ **02. Programas maiores.**
- ✓ **04. Não é recomendável para qualquer tipo de problema.**
- ✓ **08. Recursos como herança e polimorfismo podem ser desafiadores de compreender no início.**
- 16. Menor produtividade e maior custo de desenvolvimento.

RESPOSTA 15 (01 + 02 + 04 + 08)

ATENÇÃO, esta é a pegadinha mais provável da prova: o material lista "programas maiores" como CONTRAPARTIDA do POO. Maior produtividade e MENOR custo estão entre os objetivos, não entre as contrapartidas — por isso a 16 é falsa.

B03 somatória

Sobre os objetivos do paradigma orientado a objetos, some os números das afirmativas CORRETAS:

- ✓ **01. Melhorar a compreensão do sistema.**
- ✓ **02. Alto grau de reutilização por meio de componentes acopláveis.**
- ✓ **04. Facilidade de manutenção e de evolução.**
- ✓ **08. Maior produtividade e menor custo.**
- 16. Eliminar por completo a necessidade de testes de software.

RESPOSTA 15 (01 + 02 + 04 + 08)

Os cinco objetivos do material são compreensão, reutilização, manutenção, evolução, qualidade e produtividade com menor custo. Nada disso dispensa testar.

B04 assinalar

Para calcular o IMC de Marta, o material lista sete características dela (nome, idade, altura, peso, cor preferida, signo e naturalidade), mas modela apenas altura e peso. Esse princípio se chama:

- a) Encapsulamento.
- b) Sobrecarga.
- ✓ c) **Abstração: modelar somente o que é relevante para o problema.**
- d) Herança.
- e) Polimorfismo.

RESPOSTA c

Abstração é escolher, entre as muitas formas de caracterizar algo do mundo real, apenas as características úteis ao problema que se quer resolver.

B05 assinalar

A classe Cachorro tem os atributos Nome, Raça, Cor e Peso, e dela foram criados o objeto 1 (pingo, labrador, branco, 12 kg) e o objeto 2 (brutus, pastor alemão, marrom, 21 kg). Sobre esses dois objetos, assinale a alternativa correta:

- ✓ a) **Compartilham a mesma estrutura, definida pela classe, mas possuem estados diferentes.**
- b) São a mesma instância, pois pertencem à mesma classe.
- c) Possuem estruturas diferentes, pois têm valores diferentes.
- d) Não podem existir ao mesmo tempo na memória.
- e) Compartilham os valores dos atributos, pois a classe é única.

RESPOSTA a

A classe define a estrutura, ou seja, quais atributos e operações existem. Cada objeto tem seu próprio estado, que é o valor desses atributos.

B06 relacionar colunas

Relacione cada termo à definição de Barker (2005) apresentada no material:

Coluna da esquerda: A. Objeto · B. Classe · C. Estado · D. Operação

- (C) Corresponde ao valor dos atributos e pode mudar ao longo do tempo.
- (A) Construção de software que empacota estado e comportamento e representa uma abstração do mundo real.
- (D) Ação que o objeto é capaz de executar, podendo usar os dados do próprio objeto.
- (B) Abstração que descreve as características comuns de um grupo de objetos; funciona como molde.

RESPOSTA C - A - D - B

Objeto empacota estado e comportamento; classe é o molde; estado é o valor dos atributos; operação é o que o objeto sabe fazer.

UML: diagrama de classes e diagrama de objetos

B07 assinalar

Sobre o diagrama de objetos, assinale a alternativa INCORRETA:

- a) Mostra uma fotografia do estado detalhado do sistema num instante do tempo.
- b) Contém somente objetos, não contém classes.
- c) O nome do objeto aparece sublinhado, no formato nomeObjeto : Classe.
- ✓ d) **Representa tanto a estrutura de dados quanto o comportamento dos objetos.**
- e) Permite representar um objeto anônimo, omitindo o nome antes dos dois-pontos.

RESPOSTA d

O material diz explicitamente que o diagrama de objetos tem uso limitado justamente porque apresenta somente estruturas de dados, o estado, SEM representar o comportamento.

B08 relacionar colunas

Relacione o que aparece em cada diagrama, tomando a classe Pessoa como exemplo:

Coluna da esquerda: A. Diagrama de classes · B. Diagrama de objetos

- (B) marta : Pessoa com peso = 78 e altura = 1.71
- (A) Pessoa com peso : double e calcularImc() : double
- (B) Vários retângulos da mesma classe, cada um com seu próprio estado.
- (A) Um único retângulo, com três compartimentos: nome, atributos e operações.

RESPOSTA B - A - B - A

O diagrama de classes descreve os tipos (peso : double); o de objetos mostra instâncias com valores (peso = 78). Os dois são complementares.

B09 somatória

Sobre a notação do diagrama de objetos, some os números das afirmativas CORRETAS:

- ✓ 01. Cada objeto é representado por um retângulo com dois compartimentos.
- ✓ 02. No topo fica a identificação no formato nomeObjeto : Classe.
- ✓ 04. O nome do objeto é sublinhado, e é isso que o distingue de uma classe no diagrama.
- ✓ 08. Abaixo da identificação ficam os valores dos atributos.

16. É obrigatório nomear todos os objetos; objetos anônimos não são permitidos.

RESPOSTA 15 (01 + 02 + 04 + 08)

Objeto anônimo é permitido: basta omitir o nome e escrever : Classe.

B10 relacionar colunas

Relacione cada símbolo de visibilidade da UML ao seu significado:

Coluna da esquerda: A. - · B. + · C. # · D. ~

- (C) Protegido; será estudado mais tarde, junto com herança.
- (A) Privado: somente visível pela própria classe.
- (D) De pacote; corresponde à ausência de palavra reservada em Java.
- (B) Público: visível para qualquer classe.

RESPOSTA C - A - D - B

A tabela do material tem quatro símbolos. O ~ (de pacote) é o menos lembrado e corresponde a não escrever modificador nenhum em Java.

B11 assinalar

Numa associação entre Pessoa e Livro, a Pessoa recebe o papel - autor e o Livro o papel - obra. Perguntando "dado um autor, quantas obras ele pode escrever?" e "dada uma obra, quantos autores pode ter?", as multiplicidades corretas são:

- a) 1 do lado do autor e 1 do lado da obra.
- ✓ b) 0..* do lado da obra e 1..* do lado do autor.
- c) 1..* do lado da obra e 0..1 do lado do autor.
- d) 0..1 nas duas pontas.
- e) 3..5 do lado da obra e 1 do lado do autor.

RESPOSTA b

Um autor pode não ter escrito nenhuma obra e não há teto, logo 0..* no lado das obras. Uma obra precisa de pelo menos um autor, logo 1..* no lado dos autores. A multiplicidade é sempre escrita na ponta OPOSTA àquela que você fixou na pergunta.

Implementação, memória e o operador new

B12 somatória

O operador new realiza quatro operações. Some os números das afirmativas CORRETAS:

- ✓ 01. Cria o objeto na memória, alocando espaço para as variáveis de instância.
- ✓ 02. Inicializa as variáveis de instância com os valores padrão.
- ✓ 04. Executa o construtor que foi utilizado no próprio new.
- ✓ 08. Retorna o endereço de memória do objeto criado.

16. Libera automaticamente a memória do objeto anterior de mesmo tipo.

RESPOSTA 15 (01 + 02 + 04 + 08)

São exatamente as quatro operações listadas no material. Liberar memória é trabalho do coletor de lixo, não do new.

B13 assinalar

Considere o trecho `ContaBancaria conta1 = new ContaBancaria();`. Sobre o que a variável `conta1` armazena, assinale a alternativa correta:

- ✓ a) Armazena o endereço de memória do objeto, não o objeto em si.
- b) Armazena uma cópia de todos os atributos do objeto.
- c) Armazena o objeto inteiro, dentro da área Stack.
- d) Não armazena nada até o primeiro setter ser chamado.
- e) Armazena o nome da classe como texto.

RESPOSTA a

Variável de referência guarda endereço. O objeto em si fica no Heap; a variável fica no Stack.

B14 assinalar

Uma classe declara `public ContaBancaria(String titular)` e nenhum outro construtor. O que acontece ao escrever `new ContaBancaria()`?

- a) Compila normalmente, pois o construtor padrão sempre existe.
- ✓ b) Não compila: ao declarar qualquer construtor, o compilador deixa de fornecer o construtor padrão sem argumentos.
- c) Compila, mas lança exceção em tempo de execução.
- d) Compila e inicializa `titular` com string vazia.
- e) Não compila, porque construtor não pode receber parâmetro.

RESPOSTA b

O construtor padrão só é fornecido quando NENHUM construtor foi implementado. Declarou um, perdeu o de graça — se quiser os dois, precisa declarar os dois.

B15 relacionar colunas

Relacione cada elemento do código ao seu equivalente no diagrama de classes:

Coluna da esquerda: A. Variável de instância · B. Método · C. Membro sublinhado · D. Tipo de retorno void

- (B) Operação da classe, escrita com parênteses após o nome.
- (C) Membro de classe (static), que pertence à classe e não à instância.
- (A) Atributo da classe, escrito como `nome : tipo`.
- (D) Operação que altera o estado e não devolve valor (procedimento).

RESPOSTA B - C - A - D

Atributo vira variável de instância, operação vira método, membro estático aparece sublinhado e quem não devolve nada é void.

Escopo de variáveis e inicialização

B16 assinalar

Considere a classe de referência do material, com `var1` declarada no corpo da classe, `var2` dentro de `metodoB` e `var3` dentro de um bloco do `{ }`. Assinale a alternativa correta:

- a) `var3` continua existindo depois do `}` que fecha o bloco do `do/while`.
- b) `var2` pode ser lida por `metodoA`.
- ✓ c) **`var1` pode ser lida e alterada tanto por `metodoA` quanto por `metodoB`.**
- d) `var1` só existe dentro do método em que for usada pela primeira vez.
- e) As três variáveis têm o mesmo escopo, pois estão na mesma classe.

RESPOSTA C

O escopo da variável de instância são TODOS os métodos da classe. A local vale só no seu método e a de bloco morre no `}` do bloco.

B17 relacionar colunas

Relacione cada tipo de variável de instância ao seu valor padrão em Java:

Coluna da esquerda: A. `int` · B. `double` · C. `boolean` · D. Referência (objeto, `String`, `array`)

- (C) `false`
- (D) `null`
- (A) `0`
- (B) `0.0`

RESPOSTA C - D - A - B

Númerica inteira começa em `0`, ponto flutuante em `0.0`, lógica em `false` e referência em `null`. O `char` começa em `\u0000`, o caractere nulo.

B18 assinalar

Sobre variáveis locais e de bloco em Java, assinale a alternativa correta:

- a) Recebem valor padrão igual ao das variáveis de instância.
- ✓ b) **Não possuem valor inicial; ler antes de atribuir gera erro de compilação.**
- c) Não possuem valor inicial; ler antes de atribuir gera lixo de memória em tempo de execução.
- d) São sempre inicializadas com `null`, independentemente do tipo.
- e) Só podem ser declaradas no início do método.

RESPOSTA b

O material é explícito: tentar ler antes de atribuir gera ERRO DE COMPILAÇÃO, não lixo. Guarde isso, porque é o ponto que a questão 16 da L07 confunde.

B19 assinalar

No exemplo de sombreamento do material, `var1` de instância vale 20, a local é declarada como `int var1 = 5`; e o método executa `this.var1 = 10`;. O que imprimem, nessa ordem, `System.out.println(var1)`; e `System.out.println(this.var1)`;

- ✓ a) **5 e 10**
- b) 10 e 5
- c) 20 e 10
- d) 5 e 20
- e) 10 e 10

RESPOSTA a

`var1` sem qualificador é a LOCAL, que vale 5. `this.var1` é a de instância, que acabou de ser alterada para 10. A linguagem sempre dá preferência à variável de menor escopo.

B20 somatória

Sobre boas práticas de escopo apontadas no material, some os números das afirmativas CORRETAS:

- ✓ 01. Declarar cada variável no menor escopo possível e perto de onde é usada.
 - ✓ 02. Preferir variáveis locais quando o dado não precisa persistir no objeto.
 - ✓ 04. Não confiar no valor padrão de variáveis de instância como se fosse valor de negócio.
 - ✓ 08. Evitar sobreamento desnecessário, exceto no padrão `this.campo = campo de construtores e setters`.
16. Declarar todas as variáveis como de instância, para facilitar o reuso entre métodos.

RESPOSTA 15 (01 + 02 + 04 + 08)

A 16 é o oposto da primeira boa prática: o material manda usar o MENOR escopo possível.

Encapsulamento

B21 assinalar

No exemplo da motivação do encapsulamento, a classe cliente executa `conta1.saldo = 10000;`. Qual é exatamente o problema?

- a) O comando não compila, pois `saldo` é `double`.
- ✓ b) Define um saldo sem que exista um depósito correspondente, violando a integridade dos dados.
- c) O saldo fica correto, mas o desempenho piora.
- d) O problema só existe se a classe não tiver construtor.
- e) Nenhum: acessar atributo diretamente é a forma recomendada.

RESPOSTA b

O estado do objeto passou a ser alterado por fora, sem passar pela regra de negócio. Por isso o atributo precisa ser `private` e o acesso, controlado por métodos.

B22 somatória

Sobre getters e setters conforme o material, some os números das afirmativas CORRETAS:

- ✓ 01. O getter nunca tem parâmetro, é do tipo função e retorna dado do mesmo tipo do atributo.
 - ✓ 02. O setter sempre tem um parâmetro, do mesmo tipo do atributo, e é do tipo procedimento.
 - ✓ 04. Para atributo lógico, o getter usa o prefixo `is` em vez de `get`.
 - ✓ 08. O setter é o lugar natural para validar o valor antes de alterar o estado do objeto.
16. Um setter que só faz a atribuição direta já agrega proteção sobre o dado.

RESPOSTA 15 (01 + 02 + 04 + 08)

O material diz o contrário da 16: um setter que só atribui NÃO agrega proteção nenhuma. A proteção vem da validação.

B23 assinalar

Sobre as boas práticas de encapsulamento, assinale a alternativa correta:

- ✓ a) Somente é admissível utilizar `public` para constantes.
- b) Todo atributo deve ser `public` para permitir reuso.
- c) Getters e setters devem ser criados para todos os atributos, sem exceção.
- d) Atributos `private` dispensam métodos de acesso.
- e) O encapsulamento se aplica apenas a métodos, nunca a atributos.

RESPOSTA a

A regra do material é clara: encapsular todos os atributos, e `public` só se justifica para constantes. Getter e setter entram apenas quando há necessidade de expor ou de permitir alteração.

B24 assinalar

O material compara o encapsulamento de métodos com um motorista que liga o carro sem conhecer o mecanismo interno de combustão. O que essa analogia ilustra?

- a) Que métodos devem sempre ser privados.
- ✓ **b) Que é possível ocultar a complexidade interna do objeto, criando uma forma simplificada de usá-lo, o que favorece a reutilização.**
- c) Que o objeto não deve ter comportamento, apenas dados.
- d) Que toda classe precisa de um construtor sem argumentos.
- e) Que atributos devem ser acessados diretamente para ganhar desempenho.

RESPOSTA b

Encapsular método é esconder o "como" e expor só o "o quê". Quem usa a classe não precisa saber o que acontece por dentro.

B25 relacionar colunas

Relacione cada elemento ao seu papel no código `public void setTitular(String titular) { this.titular = titular; }`:

Coluna da esquerda: A. `this.titular` · B. `titular` (sem qualificador) · C. `void` · D. `public`

- (B) O parâmetro recebido pelo método, que tem menor escopo e por isso tem preferência.
- (D) Modificador de acesso que torna o método visível para qualquer classe.
- (A) A variável de instância, acessada explicitamente pelo objeto corrente.
- (C) Indica que o método é um procedimento e não devolve valor.

RESPOSTA B - D - A - C

`this` refere-se ao objeto corrente. Sem ele, o Java usaria o identificador de menor escopo, que é o parâmetro, e a atribuição não faria nada.

Membros de classe, sobrecarga e construtores

B26 somatória

Sobre as restrições dos métodos de classe (`static`), some os números das afirmativas CORRETAS:

- ✓ **01. Podem manipular variáveis de classe.**
- ✓ **02. Não podem manipular variáveis de instância sem uma instância explícita.**
- ✓ **04. Não podem reusar métodos de instância.**
- ✓ **08. Não podem utilizar a palavra `this`.**

16. Só podem ser chamados depois que pelo menos um objeto da classe for criado.

RESPOSTA 15 (01 + 02 + 04 + 08)

A 16 nega a própria razão de existir do `static`: membro de classe pode ser usado SEM que haja instância alguma.

B27 assinalar

Assinale a alternativa que contém apenas membros estáticos da biblioteca Java citados no material:

- ✓ **a) `Integer.MAX_VALUE`, `Math.sqrt()`, `Math.abs()`, `Math.max()`, `JOptionPane.showInputDialog()`**
- b) `conta.getSaldo()`, `conta.sacar()`, `new Scanner()`
- c) `this.saldo`, `this.titular`, `super.metodo()`
- d) `try`, `catch`, `finally`, `throw`
- e) `private`, `public`, `protected`, `static`

RESPOSTA a

Todos os da alternativa (a) são acessados por `Classe.membro`, sem criar objeto. As outras listam métodos de instância, referências ou palavras reservadas.

B28 assinalar

Considere os métodos `int somar(int a, int b)`, `double somar(double a, double b)` e `int somar(int a, int b, int c)` na mesma classe. Sobre esse código, assinale a alternativa correta:

- a) Não compila, pois não é permitido repetir o nome do método.
- ✓ **b) É sobrecarga válida: mesmo nome, listas de parâmetros diferentes.**
- c) É sobrecarga válida apenas porque os tipos de retorno são diferentes.
- d) Só os dois primeiros formam sobrecarga; o terceiro é inválido.
- e) É sobrescrita de métodos, não sobrecarga.

RESPOSTA b

O que diferencia é a assinatura, isto é, a lista de parâmetros. O tipo de retorno não entra nessa conta — tanto que dois métodos com a MESMA assinatura e retornos diferentes não podem coexistir.

B29 relacionar colunas

Relacione cada característica ao conceito correto:

Coluna da esquerda: A. Construtor · B. Método comum · C. Variável de classe · D. Método de classe

- (A) Não possui tipo de dado de retorno e tem o mesmo identificador da classe.
- (C) É comum e compartilhada entre todas as instâncias; aparece sublinhada na UML.
- (D) Pode ser chamado por `Classe.metodo()` e não pode usar `this`.
- (B) Declara um tipo de retorno e é invocado sobre um objeto já existente.

RESPOSTA A - C - D - B

O construtor é o único que não declara tipo de retorno. `static` é o que dá o comportamento de classe, sublinhado no diagrama.

B30 somatória

Sobre construtores, some os números das afirmativas CORRETAS:

- ✓ **01. São invocados exclusivamente durante a criação de objetos.**
- ✓ **02. Servem para inicializar o objeto.**
- ✓ **04. Sua declaração é semelhante à de um método, porém sem tipo de dado de retorno.**
- ✓ **08. Ao declarar qualquer construtor, o compilador deixa de fornecer o construtor padrão.**
- 16. É obrigatório declarar pelo menos um construtor em toda classe.

RESPOSTA 15 (01 + 02 + 04 + 08)

Não é obrigatório criar construtor: sem nenhum, o compilador fornece o padrão sem argumentos. Mas assim que você declara um, perde esse padrão.

Lançamento de exceções

B31 assinalar

Sobre o efeito de uma exceção lançada e não tratada, assinale a alternativa correta:

- a) O programa continua normalmente, ignorando a linha problemática.
- ✓ **b) O método cria um objeto de exceção que contém informação sobre o erro, notifica o sistema e a execução do programa é abortada.**
- c) O compilador se recusa a gerar o arquivo `.class`.
- d) O valor inválido é gravado assim mesmo, mas com um aviso no console.
- e) A exceção é convertida automaticamente em valor de retorno `null`.

RESPOSTA b

Lançar exceção é criar o objeto que caracteriza o erro e notificar o sistema. Sem tratamento, o efeito é abortar a execução.

B32 assinalar

Considere o método abaixo:

```
public void metodo() {  
    throw new IllegalArgumentException("sempre lança");  
    salario = 10;  
}
```

Assinale a alternativa correta:

- a) Compila e nunca executa a atribuição.
- ✓ **b) Não compila: a linha após um throw incondicional é código inalcançável.**
- c) Compila, e a atribuição executa antes do lançamento.
- d) Não compila, porque IllegalArgumentException precisa de throws na assinatura.
- e) Compila apenas se o método declarar throws IllegalArgumentException.

RESPOSTA b

Quando o throw é SEMPRE executado, nada pode vir depois dele no mesmo bloco. Diferente é o throw dentro de um if: aí a linha seguinte é alcançável no caso normal.

B33 assinalar

Por que IllegalArgumentException é a exceção usada nos setters e construtores do material?

- a) Porque é a única exceção que existe em Java.
- ✓ **b) Porque é uma exceção pronta da biblioteca, usada para indicar que um argumento recebido é inválido.**
- c) Porque é uma exceção verificada e obriga o chamador a tratá-la.
- d) Porque impede que o programa seja abortado.
- e) Porque só pode ser usada dentro de blocos try.

RESPOSTA b

Ela é subclasse de RuntimeException, portanto NÃO é verificada — o compilador não obriga a tratar. O nome comunica exatamente a intenção: o argumento recebido não serve.

B34 relacionar colunas

Relacione cada palavra ao seu papel no mecanismo de exceções:

Coluna da esquerda: A. throw · B. throws · C. try · D. catch

- (D) Recebe o objeto da exceção e contém os comandos executados quando o erro ocorre.
- (A) Cria e lança o objeto de exceção, interrompendo o método na hora.
- (C) Delimita o bloco cuja execução passa a ser monitorada.
- (B) Aparece na assinatura do método e delega o tratamento ao chamador.

RESPOSTA D - A - C - B

O par que mais confunde é throw (lança, dentro do corpo) e throws (declara, na assinatura). Repare no s.

Tratamento de exceções

B35 relacionar colunas

Relacione cada classe da hierarquia de erros ao seu lugar:

Coluna da esquerda: A. Throwable · B. Error · C. Exception · D. RuntimeException

- (B) Erros graves como falta de memória e estouro de pilha; não se espera que sejam tratados.
- (A) Raiz da hierarquia; qualquer classe de erro precisa estender dela.
- (D) Subclasse de Exception que NÃO é verificada pelo compilador.
- (C) Condições que o programa poderia contornar e que podem e devem ser tratadas.

RESPOSTA B - A - D - C

Throwable é a raiz, com dois ramos: Error (não trate) e Exception (trate). Dentro de Exception, o ramo RuntimeException é o não verificado.

B36 assinalar

Considere o laço for (`int i = 0; i <= nota.length; i++`) percorrendo um vetor `double[] nota = new double[3]`. Segundo a classificação do material, esse é um exemplo de:

- a) Erro de execução, impossível de evitar.
- ✓ **b) Erro de lógica: erro na concepção do algoritmo, que deve ser evitado.**
- c) Erro do compilador Java.
- d) Exceção verificada, que exige throws.
- e) Erro de sintaxe, detectado antes de rodar.

RESPOSTA b

O programa compila e roda, mas o raciocínio está errado: índices válidos vão de 0 a length-1. Erros de lógica nascem do programador e devem ser evitados, ao contrário dos de execução, que muitas vezes dependem do usuário ou do ambiente.

B37 somatória

Sobre como o mecanismo escolhe qual cláusula catch executar, some os números das afirmativas CORRETAS:

- ✓ **01. Ao ocorrer o erro, é criado um objeto da classe que o caracteriza e o programa é notificado.**
 - ✓ **02. O comando que causou o erro é interrompido.**
 - ✓ **04. As cláusulas catch são verificadas na ordem em que foram escritas.**
 - ✓ **08. Encontrada uma cláusula compatível, o fluxo segue nela e, ao terminar, prossegue após o comando try.**
16. Todas as cláusulas catch compatíveis são executadas, uma após a outra.

RESPOSTA 15 (01 + 02 + 04 + 08)

Só UMA cláusula catch executa: a primeira compatível encontrada na ordem de escrita. Por isso Exception fica por último, como "pega tudo".

B38 assinalar

Sobre a regra de quais exceções são verificadas (checked) pelo compilador, assinale a alternativa correta:

- a) Todas as subclasses de Throwable são verificadas.
- ✓ **b) Error e RuntimeException, e todas as suas subclasses, NÃO são verificadas; as demais são.**
- c) Apenas Error e suas subclasses são verificadas.
- d) Somente exceções criadas pelo programador são verificadas.
- e) Nenhuma exceção é verificada a partir do Java 7.

RESPOSTA b

Essa é a regra literal do material. Consequência prática: `IllegalArgumentException` e `NullPointerException` não obrigam a nada, mas `FileNotFoundException` obriga `try..catch` ou `throws`.

B39 somatória

Sobre a propagação de erros, some os números das afirmativas CORRETAS:

- ✓ 01. Quando um método é interrompido por um erro, o controle retorna ao método que o chamou.
- ✓ 02. Se o chamador também não puder tratar o erro, ele também é interrompido.
- ✓ 04. A propagação sobe pela cadeia de chamadas até que algum método consiga tratar o erro.
- ✓ 08. O último método que tem a oportunidade de tratar o erro é o `main()`.

16. Se nem o `main()` tratar o erro, o Java escolhe automaticamente um tratamento padrão e o programa continua.

RESPOSTA 15 (01 + 02 + 04 + 08)

Se nem o `main()` trata, o programa é interrompido. Não existe tratamento automático de resgate.

B40 assinalar

Uma variável é declarada dentro do bloco `try`. Sobre usá-la no `catch`, assinale a alternativa correta:

- a) Funciona normalmente, pois `try` e `catch` compartilham o mesmo escopo.
- ✓ b) **Não funciona: `try` e `catch` são blocos distintos e cada um define seu próprio escopo de variáveis.**
- c) Funciona apenas se a variável for `static`.
- d) Funciona apenas se a variável for inicializada com `null`.
- e) Funciona, mas o valor sempre chega como `null` ao `catch`.

RESPOSTA b

Cada bloco define escopo próprio. Se precisar do valor nos dois lugares, declare a variável ANTES do `try`.

B41 assinalar

A partir de qual versão do Java uma única cláusula `catch` pode tratar várias classes de erro, e com qual operador?

- a) Java 5, com o operador `&`.
- ✓ b) **Java 7, com o operador `|`.**
- c) Java 8, com o operador `||`.
- d) Java 11, com vírgula.
- e) Não é possível em nenhuma versão.

RESPOSTA b

O multi-catch existe desde o Java 7 e usa uma barra vertical: `catch (IOException | SQLException ex)`. Serve quando o tratamento é o mesmo.

Qualidade de software e testes de unidade

B42 relacionar colunas

Relacione a fase em que o erro é detectado ao seu custo relativo de correção (IBM, 2008):

Coluna da esquerda: A. 1 · B. 5 · C. 15 · D. 30

- (B) Implementação
- (D) Em produção
- (A) Especificação e arquitetura
- (C) Testes de sistema

RESPOSTA B - D - A - C

A progressão é 1, 5, 15, 30. Um erro que custaria 1 na especificação pode custar 30 em produção — é o argumento central a favor de testar cedo.

B43 somatória

Sobre o plano de testes, some os números das afirmativas CORRETAS:

- ✓ **01. Testes precisam ser planejados, e esse planejamento é registrado formalmente num documento.**
- ✓ **02. O plano pode conter propósito, identificação, itens a testar, critérios de aceite, documentos produzidos e ambiente.**
- ✓ **04. Dentro do plano há um conjunto de casos de teste.**
- ✓ **08. Cada caso de teste descreve um teste particular, com entrada própria e saída esperada.**

16. Um bom plano de testes dispensa a execução dos testes.

RESPOSTA 15 (01 + 02 + 04 + 08)

O plano organiza; quem encontra o erro é a execução. A estrutura Caso / Descrição / Entrada / Saída esperada é a que vira código no JUnit.

B44 relacionar colunas

Relacione cada método assert à sua descrição:

Coluna da esquerda: A. assertEquals(esperado, real) · B. assertEquals(esperado, real, limite) · C. assertEqualsNotNull(objeto) · D. assertEqualsThrows(Classe, lambda)

- (B) Compara decimais respeitando uma margem de diferença, por causa de arredondamento.
- (D) Verifica se o trecho de código lança a exceção esperada.
- (A) Verifica se o valor esperado é igual ao obtido.
- (C) Verifica se a variável referencia um objeto.

RESPOSTA B - D - A - C

A versão de três parâmetros é só para double: comparar decimal por igualdade exata é problemático, então aceita-se uma tolerância (delta).

B45 assinalar

Por que o segundo argumento de assertEquals precisa ser uma expressão lambda () -> ... e não a chamada direta do método?

- a) Porque lambda é mais legível.
- ✓ **b) Porque, sem o lambda, a chamada seria avaliada antes de chegar ao assertEquals e o código nem compila.**
- c) Porque assertEquals só aceita métodos static.
- d) Porque o lambda converte a exceção em valor booleano.
- e) Porque o JUnit exige lambda em todos os asserts.

RESPOSTA b

Com o lambda, quem executa o trecho é o próprio assertEquals, e aí ele consegue capturar e conferir a exceção.

B46 assinalar

Você precisa de um objeto novo e limpo antes de CADA caso de teste. Qual anotação usar, e por quê?

- a) @BeforeAll, porque roda uma vez e é mais eficiente.
- ✓ **b) @BeforeEach, porque roda antes de cada método de teste e garante estado independente.**
- c) @AfterEach, porque limpa o estado entre os testes.
- d) @Test, porque já inicializa o contexto automaticamente.
- e) @Order(1), porque garante que a inicialização venha primeiro.

RESPOSTA b

@BeforeAll roda uma única vez para a classe inteira e por isso exige método static. Reserve-a para preparações caras, como abrir uma conexão única.

B47 relacionar colunas

Relacione cada anotação do JUnit à sua funcionalidade:

Coluna da esquerda: A. @Disabled · B. @DisplayName · C. @Timeout · D. @Tag

- (B) Define um nome legível para o teste, facilitando a leitura dos relatórios.
- (D) Agrupa testes, permitindo executar subconjuntos.
- (A) Desabilita temporariamente um teste ou classe.
- (C) Define tempo máximo de execução; se excedido, o teste falha.

RESPOSTA B - D - A - C

Todas aparecem na tabela de anotações do material. @Order(n) só funciona se a classe tiver @TestMethodOrder.

B48 assinalar

Sobre a organização das classes de teste, assinale a alternativa correta:

- a) Uma única classe de teste deve cobrir todas as classes do sistema.
- ✓ **b) Há uma classe de teste paralela para cada classe testada, geralmente com o sufixo Test.**
- c) A classe de teste deve ficar dentro da pasta src, junto com a classe testada.
- d) Os métodos de teste devem retornar boolean indicando sucesso.
- e) A classe de teste precisa estender a classe testada.

RESPOSTA b

Calculadora gera CalculadoraTest. A pasta test fica FORA de src e é marcada como pasta de fontes de teste na IDE.

Associações, multiplicidade e ArrayList

B49 assinalar

Sobre a navegabilidade de uma associação, assinale a alternativa correta:

- a) Por padrão a navegação é unidirecional, e a seta indica que virou bidirecional.
- ✓ **b) Por padrão a navegação é bidirecional, e a seta limita a navegação a uma única direção.**
- c) A navegabilidade não pode ser representada no diagrama de classes.
- d) A seta sempre aponta para a classe que contém a chave primária.
- e) Navegabilidade e multiplicidade são o mesmo adorno.

RESPOSTA b

O padrão é bidirecional. Desenhar a seta é RESTRINGIR: com a seta de Livro para Pessoa, do livro chega-se aos autores, mas da pessoa não se chega às obras.

B50 relacionar colunas

Relacione cada multiplicidade ao seu significado:

Coluna da esquerda: A. 0..1 · B. 1 · C. 0..* · D. 1..*

- (D) Há pelo menos um objeto envolvido, podendo haver muitos.
- (C) Pode ou não haver instâncias participando do relacionamento.
- (B) Exatamente um objeto se relaciona com os objetos da outra classe.
- (A) Não precisam estar relacionados; havendo relacionamento, no máximo uma instância.

RESPOSTA D - C - B - A

A diferença que mais cai: 0..* admite nenhum, 1..* exige pelo menos um. Existe ainda a forma m..n, como 3..5.

B51 assinalar

A classe `Funcionario` se associa a si mesma pela associação `chefia` ►, com os papéis - gerente e - empregado. Esse tipo de associação se chama:

- a) Associação bidirecional.
- ✓ **b) Associação reflexiva: conecta objetos de uma mesma classe.**
- c) Herança.
- d) Composição.
- e) Dependência circular.

RESPOSTA b

Reflexiva é a associação de uma classe com ela mesma. Como a mesma classe aparece nas duas pontas, são os PAPÉIS que distinguem os lados.

B52 somatória

Sobre o `ArrayList`, some os números das afirmativas CORRETAS:

- ✓ **01. Uma instância de `ArrayList` pode armazenar diversos objetos.**
- ✓ **02. Não possui tamanho limitado, como o vetor.**
- ✓ **04. Não armazena dados primitivos, apenas objetos.**
- ✓ **08. Os objetos armazenados são recuperados pela posição.**

16. Dispensa `import`, por fazer parte do pacote `java.lang`.

RESPOSTA 15 (01 + 02 + 04 + 08)

É preciso `import java.util.ArrayList;`. Quem dispensa `import` é o pacote `java.lang`, onde ficam `String`, `Math` e `Integer`.

B53 relacionar colunas

Relacione cada método do `ArrayList` à sua descrição:

Coluna da esquerda: A. `add(E)` · B. `get(int)` · C. `remove(Object)` · D. `size()`

- (D) Retorna a quantidade de objetos armazenados.
- (A) Guarda um objeto na coleção.
- (B) Obtém o objeto na posição indicada.
- (C) Remove o objeto da coleção.

RESPOSTA D – A – B – C

Cuidado com `remove`: numa lista de `Integer`, `remove(1)` remove a POSIÇÃO 1, porque o Java escolhe a versão `remove(int)`.

B54 assinalar

No trecho `ArrayList<Aluno> turma = new ArrayList<>();`, o que é o `<>` do lado direito?

- a) Um erro de digitação: o tipo deveria ser repetido.
- ✓ **b) O operador diamante, que repete o tipo já declarado do lado esquerdo.**
- c) Um operador de comparação.
- d) A indicação de que a lista aceita qualquer tipo.
- e) Uma anotação do compilador.

RESPOSTA b

O diamante evita repetir `<Aluno>`. A lista continua tipada: só aceita `Aluno`.

B55 somatória

Ao traduzir para código uma associação com multiplicidade acima de 1, some os números das afirmativas CORRETAS:

- ✓ 01. Cria-se uma variável na classe de origem, com identificador igual ao papel.
- ✓ 02. Na ausência de papel, o nome é derivado do nome da classe de destino, geralmente no plural.
- ✓ 04. O tipo da variável deve ser ArrayList, com o operador diamante e o nome da classe de destino.
- ✓ 08. A instância do ArrayList é criada na declaração da variável ou no construtor.

16. Deve-se criar um setter para a variável que mantém a associação.

RESPOSTA 15 (01 + 02 + 04 + 08)

O material é explícito: NÃO existe setter para a coleção de uma associação. A inclusão e a remoção são feitas por métodos próprios, que é onde ficam as regras.

B56 assinalar

A associação entre Disciplina e Aluno é 1..50. Como o material sugere impedir a inclusão do aluno 51?

- a) Ignorando silenciosamente a chamada de incluirAluno.
- ✓ b) Verificando `alunos.size()` e lançando uma exceção quando o limite for atingido.
- c) Trocando o ArrayList por um vetor de 50 posições.
- d) Chamando `alunos.clear()` antes de incluir.
- e) Não é possível limitar uma associação em Java.

RESPOSTA b

É a ideia da aula de exceções aplicada à coleção: `if (alunos.size() == 50) throw ...`. O slide usa `RuntimeException`; `IllegalStateException` comunica melhor "coleção cheia".

B57 assinalar

Sobre o getter de uma associação com multiplicidade maior que 1, como `getAlunos()`, o material levanta um ponto de atenção. Qual é?

- a) O getter deveria ser `private`.
- ✓ b) Retornar diretamente o ArrayList interno expõe a coleção, permitindo `add` e `remove` por fora e contornando as regras.
- c) O getter deveria retornar um vetor em vez de ArrayList.
- d) O getter não pode existir em associações reflexivas.
- e) O getter precisa de um parâmetro indicando a posição.

RESPOSTA b

Quem faz `getAlunos().add(...)` pula a validação de limite que está em `incluirAluno`. A saída é devolver uma cópia da lista, como o `getSeguindo()` do Sonora faz.

B58 relacionar colunas

Relacione cada forma de percorrer uma coleção à situação em que ela é a escolha certa:

Coluna da esquerda: A. `for` com índice · B. `for-each`

- (B) Só é preciso ler cada elemento, sem saber a posição.
- (A) É preciso saber em que posição o elemento está.
- (A) É preciso remover elementos durante a varredura.
- (B) O código fica mais enxuto e não há risco de errar o limite do laço.

RESPOSTA B - A - A - B

Remover dentro de um `for-each` é o erro clássico: às vezes lança `ConcurrentModificationException`, às vezes só encerra o laço mais cedo e devolve resultado errado. Veja `ERROS_PROVA_PRATICA.md`, seção 5.1.

Gabarito — Parte II

Q	Resposta	Q	Resposta	Q	Resposta	Q	Resposta
B01	b	B16	c	B31	b	B46	b
B02	15 (01 + 02 + 04 + 08)	B17	C - D - A - B	B32	b	B47	B - D - A - C
B03	15 (01 + 02 + 04 + 08)	B18	b	B33	b	B48	b
B04	c	B19	a	B34	D - A - C - B	B49	b
B05	a	B20	15 (01 + 02 + 04 + 08)	B35	B - A - D - C	B50	D - C - B - A
B06	C - A - D - B	B21	b	B36	b	B51	b
B07	d	B22	15 (01 + 02 + 04 + 08)	B37	15 (01 + 02 + 04 + 08)	B52	15 (01 + 02 + 04 + 08)
B08	B - A - B - A	B23	a	B38	b	B53	D - A - B - C
B09	15 (01 + 02 + 04 + 08)	B24	b	B39	15 (01 + 02 + 04 + 08)	B54	b
B10	C - A - D - B	B25	B - D - A - C	B40	b	B55	15 (01 + 02 + 04 + 08)
B11	b	B26	15 (01 + 02 + 04 + 08)	B41	b	B56	b
B12	15 (01 + 02 + 04 + 08)	B27	a	B42	B - D - A - C	B57	b
B13	a	B28	b	B43	15 (01 + 02 + 04 + 08)	B58	B - A - A - B
B14	b	B29	A - C - D - B	B44	B - D - A - C		
B15	B - C - A - D	B30	15 (01 + 02 + 04 + 08)	B45	b		